

A Beginner's Guide to Undoing Mistakes in Git

Making mistakes is a normal part of learning how to code, but knowing how to fix them builds real confidence. Mastering basic Git safety commands stops small terminal panic attacks and keeps your projects clean.

What you'll learn:

- 1 Discarding Uncommitted Changes with `git restore`
- 2 Unstaging Files with `git restore --staged`
- 3 Soft Resetting Commits with `git reset --soft`
- 4 Fixing Public Mistakes Safely with `git revert`

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Discarding Uncommitted Changes with git restore

When you are trying out a new idea in a file and realize it is completely broken, you do not need to manually delete your messy code line by line. The restore command looks at the last saved version of your file and replaces your current working mess with that clean version. Think of it like pressing Ctrl+Z on your entire file back to the last save point.

```
$git restore app.js
```

Cloud & DevOps Daily

How to Undo Mistakes in Git (git reset, revert, restore)

Monday, August 10, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Unstaging Files with `git restore --staged`

We have all run `git add .` in a rush and accidentally staged temporary files, configuration secrets, or system logs that we never wanted to commit. Using the staged flag allows you to remove those files from your staging area without deleting the actual code changes you made inside them. It simply puts the files back to their modified, unstaged state.`

```
$git restore --staged secret.env
```

Cloud & DevOps Daily

How to Undo Mistakes in Git (git reset, revert, restore)

Monday, August 10, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Soft Resetting Commits with `git reset --soft`

Sometimes you make a commit a bit too early, only to realize you forgot to include a crucial file or a typo fix. A soft reset moves your commit history pointer back by one step, but it leaves all your modified code sitting safely in your staging area. This allows you to add the missing files and remake the commit properly without losing any work.

```
$git reset --soft HEAD~1
```

Cloud & DevOps Daily

How to Undo Mistakes in Git (git reset, revert, restore)

Monday, August 10, 2026

Swipe to tip 4 >> [linkedin.com/in/mramanrajsingh](https://www.linkedin.com/in/mramanrajsingh)

4

TIP 4 OF 4

Fixing Public Mistakes Safely with git revert

If you push a broken feature to a shared team repository, using a reset command can disrupt everyone else's work history. Instead, git revert creates a brand new commit that performs the exact opposite actions of the bad commit. It is the safest way to undo a mistake in a team environment because it preserves past history while fixing the current bug.

```
$git revert a1b2c3d
```


Cloud & DevOps Daily

How to Undo Mistakes in Git (git reset, revert, restore)

Monday, August 10, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always run git status before executing any reset command
- + Use git diff to review your changes before discarding them
- + Practice these commands in a dummy test repository first
- + Keep your commit messages clear so you know where to return
- + Ask a teammate for help if your terminal shows a detached HEAD state

Watch Out For

- ! Using git reset --hard without backing up your current work
- ! Running destructive reset commands on shared team branches
- ! Confusing git reset (which changes history) with git revert (which adds history)

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh